

BYTE PAIR CODE

July 15, 2026

```
[3]: import numpy as np
from collections import defaultdict

# Step 1: Initial Vocabulary

vocab = {
    "l o w </w>": 5,
    "l o w e r </w>": 2,
    "n e w e s t </w>": 6,
    "w i d e s t </w>": 3
}

# Display Initial Vocabulary

print("=====Initial Vocabulary====")
for k, v in vocab.items():
    print(k, ":", v)

# Step 2: Count Word Frequencies

freq = np.array(list(vocab.values()))

print("\nWord Frequencies :", freq)
print("Total Words :", np.sum(freq))
print("Maximum Frequency :", np.max(freq))

# Step 3: Count Adjacent Symbol Pairs

def get_pair_counts(vocab):
    pairs = defaultdict(int)

    for word, frequency in vocab.items():
```

```

        symbols = word.split()

        for i in range(len(symbols) - 1):
            pair = (symbols[i], symbols[i + 1])
            pairs[pair] += frequency

        return pairs

# Step 4: Merge Best Pair

def merge_pair(pair, vocab):
    new_vocab = {}

    bigram = " ".join(pair)
    replacement = "".join(pair)

    for word in vocab:
        new_word = word.replace(bigram, replacement)
        new_vocab[new_word] = vocab[word]

    return new_vocab

# Step 5: Perform Merges
merge_rules = []
num_merges = 10

for i in range(num_merges):

    pairs = get_pair_counts(vocab)

    if not pairs:
        break

    best_pair = max(pairs, key=pairs.get)

    merge_rules.append(best_pair)

    print("\nMerge", i + 1)
    print("Best Pair :", best_pair)
    print("Frequency :", pairs[best_pair])

    vocab = merge_pair(best_pair, vocab)

```

```

# Step 6: Final Vocabulary

final_vocab = set()

for word in vocab:
    final_vocab.update(word.split())

print("\n===== Final Vocabulary =====")
print(sorted(final_vocab))
print("Vocabulary Size :", len(final_vocab))

# Step 7: Encoder

def encode(word, merge_rules):

    tokens = list(word) + ["</w>"]

    for pair in merge_rules:

        i = 0

        while i < len(tokens) - 1:

            if tokens[i] == pair[0] and tokens[i + 1] == pair[1]:
                tokens = tokens[:i] + ["".join(pair)] + tokens[i + 2:]
            else:
                i += 1

    return tokens

# Step 8: Decoder

def decode(tokens):

    word = "".join(tokens)

    return word.replace("</w>", "")

# Step 9: Test

test_word = "newest"

```

```

encoded = encode(test_word, merge_rules)
decoded = decode(encoded)

print("\n===== Merge Rules =====")

for i, rule in enumerate(merge_rules, 1):
    print(i, ":", rule)

print("\nTest Word :", test_word)
print("Encoded :", encoded)
print("Decoded :", decoded)

print("\n===== Program Completed =====")

```

===== Initial Vocabulary =====

```

l o w </w> : 5
l o w e r </w> : 2
n e w e s t </w> : 6
w i d e s t </w> : 3

```

Word Frequencies : [5 2 6 3]

Total Words : 16

Maximum Frequency : 6

Merge 1

Best Pair : ('e', 's')

Frequency : 9

Merge 2

Best Pair : ('es', 't')

Frequency : 9

Merge 3

Best Pair : ('est', '</w>')

Frequency : 9

Merge 4

Best Pair : ('l', 'o')

Frequency : 7

Merge 5

Best Pair : ('lo', 'w')

Frequency : 7

Merge 6

Best Pair : ('n', 'e')

Frequency : 6

Merge 7
Best Pair : ('ne', 'w')
Frequency : 6

Merge 8
Best Pair : ('new', 'est</w>')
Frequency : 6

Merge 9
Best Pair : ('low', '</w>')
Frequency : 5

Merge 10
Best Pair : ('w', 'i')
Frequency : 3

===== Final Vocabulary =====
['</w>', 'd', 'e', 'est</w>', 'low', 'low</w>', 'newest</w>', 'r', 'wi']
Vocabulary Size : 9

===== Merge Rules =====
1 : ('e', 's')
2 : ('es', 't')
3 : ('est', '</w>')
4 : ('l', 'o')
5 : ('lo', 'w')
6 : ('n', 'e')
7 : ('ne', 'w')
8 : ('new', 'est</w>')
9 : ('low', '</w>')
10 : ('w', 'i')

Test Word : newest
Encoded : ['newest</w>']
Decoded : newest

===== Program Completed =====

[]:

[]: